COMP0084 Information Retrieval and Data Mining Coursework

Anonymous ACL submission

1 Task 1 (D1) - Text Statistics

Text Pre-processing. Text is lowercased and tokens are extracted as maximal alphabetic runs via `[a-z]+`. Lowercasing ensures case-insensitivity to prevent spurious vocabulary inflation from case variations (e.g. *London* and *london*); digits and punctuation are excluded as the coursework requires unigram text representations, and these are not linguistic tokens. Stemming is not applied, as it brings at most modest recall benefit for English while risking conflation of semantically distinct terms (e.g. *information* and *informant*). Stop words are retained: their removal can obscure query meaning, and the empirical effect is quantified below. The vocabulary contains $|\mathcal{V}| = 115{,}703$ unique terms over $10{,}282{,}476$ total tokens.

Term Frequency Distribution. Figure 1 plots normalised term frequency $\hat{f}(k)$ against frequency rank k on a linear scale. The heavily right-skewed distribution demonstrates that a small number of terms account for a disproportionately large fraction of tokens, while the vast majority are very rare. This is consistent with Zipf's law, which predicts that term frequency is inversely proportional to rank: a few terms dominate and frequency decays rapidly.

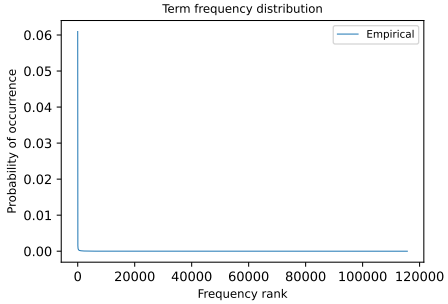

Figure 1: Empirical term frequency distribution (linear scale).

Comparison with Zipf's Law. The Zipfian distribution is:

$$f(k;\, s,\, N) = \frac{k^{-s}}{\sum_{i=1}^{N} i^{-s}} \qquad (1)$$

Setting $s = 1$, the denominator reduces to $H_N = \sum_{i=1}^{N} i^{-1} = 12.236$ for $N = 115{,}703$, giving: f(k; 1, N) = $1_{\overline{k \cdot H_N}}$

From Equation 1, the predicted frequency of the most frequent term is $\frac{1}{H_N} = 0.0817$ i.e. 8.17% of all tokens, significantly higher than the empirical frequency of $\sim$6.10%. This is a result of the fact that $H_N = O(\ln N)$: the frequency of the most common term decreases only logarithmically as vocabulary grows, while empirical top-term frequencies stabilise independently of vocabulary size. As a result, the predicted value $1/H_N$ does not decrease sufficiently to match real language usage. This is one reason why Eq. 1 tends to overestimate the frequency of high-ranked terms in practice, although the extent of this deviation is dependent on corpus composition and preprocessing choices.

Head: The primary deviation between the empirical and Zipfian distributions originates at ranks 1 and 2: *the* ($\hat{f} = 6.10\%$, predicted 8.17%, deficit -2.07%) and *of* ($\hat{f} = 3.25\%$, predicted 4.09%, deficit -0.84%). The k^{-1} term overestimates the frequency of the first-ranked term by 2.08% and the second-ranked by 0.835%, together accounting for 70.5% of total squared error of the collection. The top 1% of terms (1,157 terms) account for 99.8% of total squared error (overall MSE 6.14×10^{-9}; mean per-head-term MSE 6.13×10^{-7}, $\sim$100$\times$ the corpus-wide average). These high-ranked terms are closed-class function words, with frequencies governed by grammatical necessity rather than content. As a result, their empirical frequencies stabilise independently of corpus size, whereas the Zipfian normalisation term $1/H_N$ decreases only logarithmically with corpus size and therefore fails to accurately capture their observed frequencies.

The results indicate that while Zipf's law captures the overall shape of the distribution, it does not fully account for linguistic constraints governing high-frequency function words, leading to systematic deviations at the head.

Body: Since both distributions are normalised to unity, their pointwise differences must sum to zero:

$$\sum_{k=1}^{N}[\hat{f}(k) - f(k;\,1,\,N)] = 0 \qquad (2)$$

As such, the overestimation in the *Head* is necessarily redistributed across the lower-frequency terms. This implies that deviations from Zipf's law are structurally linked across the distribution; that is, that inaccuracies at the head are necessarily reflected in the body rather than occurring independently. The first crossover occurs between ranks 2 and 3, after which the empirical distribution lies broadly above the s = 1 prediction across a wide mid-frequency range, albeit with local oscillations.

This behaviour can be understood as a consequence of the head deficit. Since the highest-ranked terms fall below the Zipfian prediction, probability mass must be reallocated to the remaining terms, which produces a systematic surplus throughout the body. This surplus persists until approximately rank 5,829, beyond which the empirical distribution falls and remains permanently below the s=1 curve.

Since term frequencies are discrete integers, the empirical distribution forms a staircase rather than a smooth curve. This discretisation causes oscillation around the continuous $s = 1$ line, particularly in the mid-frequency region, in contrast to the smooth power-law behaviour defined by the Zipf model.

This deviation can be quantified using the Zipf formulation. Under the $s = 1$ model, the product $k \cdot f(k; 1, N)$ is constant and equal to $1/H_N = 0.0817$ for all ranks; systematic deviation from this constant therefore quantifies where and by how much the empirical distribution departs from Equation 1. Empirically, however, this product varies substantially. It is 0.061 at rank 1 (approximately 25% below prediction), rises to 0.137 at rank 1000 (approximately 67% above prediction), and then falls towards zero in the tail. Behaviour can therefore be approximately divided into three regimes, with underestimation in the head, surplus in the body, and collapse in the tail relative to the idealised Zipfian model.

Rank	Empirical	Zipf $s = 1$	Ratio	$k \cdot \hat{f}(k)$
1	6.097×10^{-2}	8.173×10^{-2}	0.746	0.061
2	3.251×10^{-2}	4.086×10^{-2}	0.796	0.065
3	2.768×10^{-2}	2.724×10^{-2}	1.016	0.083
10	8.428×10^{-3}	8.173×10^{-3}	1.031	0.084
100	8.670×10^{-4}	8.173×10^{-4}	1.061	0.087
1,000	1.368×10^{-4}	8.173×10^{-5}	1.674	0.137
10,000	6.030×10^{-6}	8.173×10^{-6}	0.738	0.060
100,000	9.730×10^{-8}	8.173×10^{-7}	0.119	0.010

Table 1: Empirical vs Zipf $s = 1$ frequency. The $k \cdot \hat{f}(k)$ column should equal $1/H_N = 0.0817$ (constant) under perfect Zipf's law.

Table 1 illustrates these deviations at representative ranks across the full vocabulary.

Tail: Zipf's law increasingly overestimates the frequency of rare terms. In the tail, staircase steps become increasingly coarse as counts approach one: each count level spans a larger portion of the rank axis, and the drops between plateaus grow proportionally. The gap widens to 0.92 $\log_{10}$ units by rank $\sim$100,000—$s = 1$ over-predicts rarest-term frequencies by nearly an order of magnitude—before the $\approx$42,000 hapax legomena produce a constant floor at $1/T \approx 9.73 \times 10^{-8}$ while $s = 1$ continues its k^{-1} decay. The tail contains proper nouns, misspellings, and domain-specific terms that inflate the vocabulary beyond what a smooth power law predicts.

The log-log plot in Figure 2 visualises these properties, with the linear relationship also confirming the power-law structure ($\log f(k) = \log C - s \cdot \log k$). However, as the log scale compresses large values and stretches small ones, tail deviations appear visually larger than those in the head despite contributing negligibly to overall error. For example, the head deficit of 2.08 percentage points produces a gap of only 0.13 $\log_{10}$ units and appears modest, while the tail staircase spans 0.92 $\log_{10}$ units despite the model only overestimating frequency by 0.0001% at this low rank.

Overall, Zipf's law provides a strong first-order approximation of term distributions, but systematic deviations – particularly at the head—highlight the limits of modelling language as a purely scale-free process.

Stop Word Removal. Removing the 127-term NLTK English stop word list reduces the vocabulary size to 115,576 terms (Figure 3), but results in a 41.47% decrease in number of tokens.

From Equation 1, removing these 127 terms changes H_N by less than 0.01% (from $12.236 \rightarrow 12.235$), leaving the $s = 1$ prediction at rank 1 es-

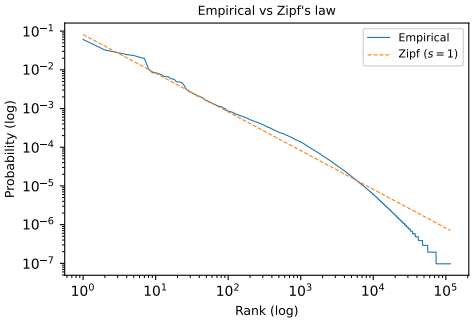

Figure 2: Empirical distribution vs. Zipf's law ($s = 1$), log-log scale.

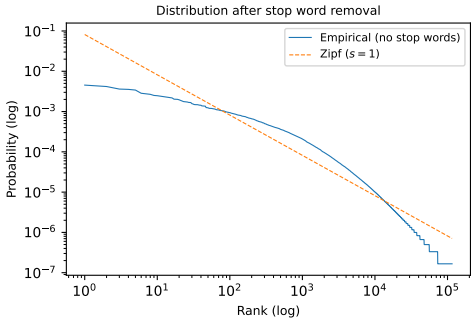

Figure 3: Distribution after stop word removal vs. Zipf's law ($s = 1$), log-log scale.

sentially unchanged ($1/H_N \approx 8.17\%$). However, the new first-ranked term *one* now has empirical frequency of only 0.45%, yielding a discrepancy of -7.72% compared to the previous -2.08% for *the* (i.e. a much larger overestimation error in magnitude). The $\log_{10}$ gap at rank 1 grows from -0.13 to -1.26, nearly a tenfold increase in log space. Overall MSE increases by a factor of 12.46, from 6.14×10^{-9} to 7.65×10^{-8}. This occurs because removing stop words eliminates the high-frequency terms that partially align with the Zipfian head, leaving a flatter empirical distribution that deviates more strongly from Equation 1. The top 1% of terms (1,155 terms) again account for 99.9% of total squared error, with mean squared error per head term increasing 12.5× to 7.65×10^{-6}.

The first crossover from below to above $s = 1$ is delayed from rank 3 to rank 77 (between *area* and *degree*): without a dominant high-frequency term, the empirical distribution stays below the Zipf prediction for the entire top-0.07% of the vocabulary. The tail structure is broadly unchanged—the hapax floor rises slightly to $1/T \approx 1.66 \times 10^{-7}$ (reflecting the reduced token count) and begins at rank 73,006, almost identical to the with-stop-words case. The fitted slope is effectively unchanged (-1.76 vs -1.77), confirming that removing 127 terms does not alter the distributional shape, only the magnitude of the head deviation.

Removing stop words worsens the agreement with Zipf's law, as it eliminates the high-frequency terms that dominate the head of the distribution and partially align with the model. However, in an information retrieval context, stop word removal may still be beneficial, as these terms carry limited semantic meaning for distinguishing relevance between documents.

2 Task 2 (D3) - Inverted Index

Construction. The inverted index is constructed over the 182,469 unique passages in `candidate-passages-top1000.tsv`, using the full 115,703-term vocabulary with stop-words retained (for the reasons above). Each passage is tokenised identically to Task 1.

The index is implemented as a Python `dict` mapping each term to a nested `dict` of {`pid`: `tf`} pairs (posting list). Dictionary lookup gives $O(1)$ average-case access per term; posting lists are sparse, storing only passages in which the term occurs. This enables retrieval by term-based lookup rather than full-corpus scanning: for a q-term query, only $\sum_t \mathrm{df}(t)$ passages are examined rather than all $N = 182{,}469$. Posting lists are stored in insertion order (by pid encounter during indexing) and no sorting by tf or pid is applied (BM25 and language model scoring require full list traversal regardless of ordering).

Stored information and justification. Five components are stored, each required by at least one downstream retrieval model.

(i) Posting lists: $t \mapsto \{p : \mathrm{tf}(t,p)\}$, where $\mathrm{tf}(t,p)$ is the raw term frequency of term t in passage p. Raw TF is stored rather than normalised TF so that the same index serves all models without committing to a single normalisation scheme at index-build time.

(ii) Document frequency: $\mathrm{df}(t) = |\{p : t \in p\}|$, the number of passages containing t. Required to compute $\mathrm{IDF}(t) = \log(N/\mathrm{df}(t))$ in TF-IDF and the corresponding factor in BM25. For example, $\mathrm{df}(the) = 157{,}771$ while $\mathrm{df}(retrieval) = 14$, reflecting the large IDF spread across the vocabulary.

(iii) Document lengths: $|p|$ for each passage p, required by three distinct computations: normalised

TF in TF-IDF ($\mathrm{tf}/|p|$), the BM25 length normalisation factor $K = k_1((1-b) + b \cdot |p|/\overline{|p|})$, and the denominator in the likelihood computation of all three language model smoothing methods. The average document length is $\overline{|p|} = 56.4$ tokens.

(iv) Collection frequency: $\mathrm{cf}(t) = \sum_p \mathrm{tf}(t,p)$, the total number of occurrences of t across all passages. This is required exclusively for Dirichlet smoothing, which uses the collection language model $P(w|\mathcal{C}) = \mathrm{cf}(w)/\sum_{w'} \mathrm{cf}(w')$. Storing cf gives $O(1)$ lookup per term; recomputing it at query time would require summing over the entire posting list. For example, $\mathrm{cf}(\textit{blood}) = 15{,}118$ while $\mathrm{df}(\textit{blood}) = 5{,}875$, indicating an average of 2.6 occurrences per passage containing the term.

(v) Corpus statistics: corpus size $N = 182{,}469$, average document length $\overline{|p|} = 56.4$, and total token count $\sum_p |p| = 10{,}282{,}476$. These scalar values are precomputed once at indexing time to enable faster query evaluation at the expense of increased storage requirements, and are used by IDF, BM25, and the collection language model respectively.

3 Task 4 (D11) – Query likelihood language models

Let $\mathrm{tf}_{w,D}$ be the term frequency of word w in passage D, $|D|$ the total number of tokens in the passage, and $|V|$ the vocabulary size (115,703). The models are scored using the natural logarithm of the joint probability:

$$\log P(Q|D) = \sum_{w \in Q} \log P(w|D). \tag{3}$$

Model Performance Expectation. Dirichlet smoothing is generally expected to outperform Laplace and Lidstone for passage retrieval. The Dirichlet estimate is defined as:

$$P(w|D)_{Dir} = \frac{tf_{w,D} + \mu P(w|C)}{|D| + \mu} \tag{4}$$

where $P(w|C)$ is the background collection probability and $\mu = 50$. The model is expected to be superior as it adapts to document length: short passages (i.e. when $|D|$ is small relative to μ) receive more weight from the collection prior $P(w|\mathcal{C})$, providing reliable estimates where per-passage term frequencies are sparse. By contrast, Laplace and Lidstone apply uniform additive smoothing regardless of document length, which penalises shorter passages through the $|D| + \epsilon|\mathcal{V}|$ denominator. Further, Dirichlet's collection prior $P(w|\mathcal{C})$ is term-specific: rare but discriminative query terms receive a meaningful background probability derived from their corpus frequency, rather than the uniform $1/|\mathcal{V}|$ used by Laplace and Lidstone.

This is confirmed empirically. For the query *"what is the county for grand rapids, mn"*, Laplace ranks first a 72-token passage about zip codes in Michigan (score -83.60), while Dirichlet selects a more focused 50-token passage about Aitkin County (score -32.73); Laplace assigns this passage only -85.19. For *"what do you do when you have a nosebleed from having your nose"*, Laplace top-ranks a 143-token passage about nasal mucus (-136.85), while Dirichlet selects a 41-token passage directly addressing nosebleed causes (-62.79), which Laplace ranks lower at -143.02. In both cases Dirichlet correctly favours shorter, more topically focused passages.

Model Similarity. Laplace and Lidstone are expected to produce the most similar rankings, given that both are additive smoothing methods with identical functional form,

$$P(w|D) = \frac{\mathrm{tf}_{w,D} + \epsilon}{|D| + \epsilon|\mathcal{V}|} \tag{5}$$

where Laplace is a specific case of Lidstone where $\epsilon = 1$. As such, both models apply a uniform prior across the entire vocabulary $|V|$ and therefore do not account for the relative frequencies of terms in the collection. This results in the relative ordering of passages being largely preserved. For example, for *"truncating meaning"*, Laplace and Lidstone produce identical top-5 rankings (scores $-20.10/-17.84$, $-20.23/-17.97$, $-20.27/-18.03$, $-20.32/-18.08$, $-20.32/-18.08$), while Dirichlet reorders them. For *"does legionella pneumophila cause pneumonia"*, the top-5 is again identical under Laplace and Lidstone but Dirichlet promotes a 48-token passage directly naming *Legionella pneumophila* (Dirichlet -22.10) above a broader 79-token passage on general pneumonia causes (-22.63).

Analysis of Lidstone Parameter ($\epsilon = 0.1$). For $\epsilon = 0.1$, the Lidstone denominator is $|D| + \varepsilon|\mathcal{V}| = 56.4 + 0.1 \times 115{,}703 \approx 11{,}627$. With an average passage length of 56.4 tokens, the actual document length contributes only $56.4/11{,}627 = 0.49\%$ of the denominator: the smoothing constant is approximately 200 times greater than the document content. As such, the model's probability scores

are dominated by the vocabulary size rather than document-specific term frequencies, reducing the model's ability to distinguish between relevant and irrelevant passages ("over-smoothing").

To avoid the document evidence being overwhelmed by the prior in this way, it is beneficial for the prior (that expected from the language) and the evidence (that in the passage) to carry approximately equal weight in the denominator, i.e. that $\epsilon|\mathcal{V}| \approx \overline{|D|}$. This gives $\varepsilon \approx 56.4/115{,}703 \approx 0.0005$, suggesting a range of $\epsilon \in [0.0001, 0.01]$ would be effective. $\epsilon = 0.1$ is too large by roughly two orders of magnitude.

Analysis of Dirichlet Parameter ($\mu = 5000$). In Dirichlet smoothing, the document's contribution to the probability estimate is scaled by $\frac{|D|}{|D|+\mu}$. For a typical 47-token passage in this collection, $\mu = 5000$ reduces the document's weight to $47/5047 = 0.9\%$, meaning that the passage content is almost entirely discarded and the score approximates $P(Q|\mathcal{C})$, the collection-wide language model, rather than $P(Q|D)$.

The resulting dominance of the collection prior is, for example, exhibited by the scores of the query *"what slows down the flow of blood"*. The top-ranked passage (47 tokens) scores -28.79 at $\mu = 50$ but -42.28 at $\mu = 5000$, reflecting almost complete loss of discriminative signal. An appropriate μ should be of similar magnitude to the average passage length; as this is 56.4 tokens here, the default $\mu = 50$ is well-calibrated to this corpus. $\mu = 5000$ would be appropriate only for a corpus of passages that averaged several thousand tokens.

4 Task 5 – Retrieval quality evaluation and machine learning

Evaluation Metrics (D13). Average Precision (AP) for a query is computed by traversing the ranked list and averaging the precision at every rank k where a relevant passage is retrieved:

$$AP = \frac{1}{|R|} \sum_{k=1}^{N} P(k) \cdot 1[d_k \in R] \qquad (6)$$

where R is the set of relevant passages and $P(k)$ is precision at rank k. Mean Average Precision (MAP) is the mean of AP across all queries.

Discounted Cumulative Gain at rank 10 (DCG@10) is defined as:

$$DCG@10 = \sum_{k=1}^{10} \frac{2^{rel(k)} - 1}{\log_2(k+1)}. \qquad (7)$$

The Ideal DCG (IDCG) is then computed by sorting documents in descending order of relevance, providing the normalised DCG@10 (NDCG@10) via:

$$NDCG@10 = \frac{DCG@10}{IDCG@10} \qquad (8)$$

For binary relevance labels $\mathrm{rel} \in \{0, 1\}$, the numerator simplifies to 1 for relevant and 0 for irrelevant passages. If $IDCG = 0$ (i.e. no relevant documents exist for a query), NDCG is defined as 0. The AP implementation handles queries with no relevant passages (AP = 0) and resolves ranking ties by retaining the original candidate order. MAP is optimised on train-data.tsv and performance is reported on `validation-data.tsv` using both MAP and NDCG@10.

BM25 Parameter Optimisation (D13). A grid search was performed over $k_1 \in \{0.5, 1.0, 1.2, 1.5, 2.0\}$ and $b \in \{0.0, 0.25, 0.50, 0.75, 1.0\}$. For each combination of hyperparameters,

(i) BM25 scores were computed for all query-document pairs,

(ii) Documents were ranked per query,

(iii) Average Precision was computed per query, and

(iv) MAP over the training set `train-data.tsv` was used as the optimisation objective.

The optimal parameters were $k_1 = 0.5$, $b = 0.25$ (training MAP $= 0.0242$). Table 2 reports validation performance.

Model	MAP	NDCG@10
BM25 default (k_1=1.2, b=0.75)	0.0193	0.0196
BM25 optimised (k_1=0.5, b=0.25)	0.0191	0.0195
LogReg (FastText, η=0.3, 3 epochs)	0.0302	0.0287

Table 2: Validation set performance for all models.

The optimised parameters improve training MAP but not validation MAP, indicating mild overfitting to the training queries. The optimal $k_1 = 0.5$ implies aggressive term frequency (TF) saturation: TF contributes less than in the default setting, which benefits queries where any single occurrence is sufficient evidence of relevance. The low $b = 0.25$ applies minimal document length normalisation, appropriate for a collection where passage lengths are short and relatively uniform ($\overline{|p|} = 56.4$ tokens). Both MAP and NDCG@10

are slightly lower for the optimised than default model on validation, confirming that the optimised parameters overfit the training queries and do not generalise to unseen queries.

The Zipf distribution established in Task 1 shapes these results: the steep frequency decay means a small number of terms have near-zero IDF while the majority have high IDF. Stop words, which account for 41.47% of tokens, receive IDF close to zero ($\mathrm{df}(the) = 157{,}771$ of 182,469 passages), naturally downweighting them without explicit removal. BM25 and TF-IDF therefore exploit the same distributional structure described by Zipf's law.

Word Embeddings and Logistic Regression (D14). Queries and passages are represented using FastText embeddings (`fasttext-wiki-news-subwords-300`, 300 dimensions), a subword-aware model trained on Wikipedia and news text. Each query or passage is embedded by averaging the FastText vectors of its tokens (using the same `[a-z]+` tokenisation as Tasks 1–4); out-of-vocabulary tokens are ignored. This yields a single 300-dimensional vector per text. The feature for a query–passage pair is the concatenation of both vectors augmented with their cosine similarity, yielding $\mathbf{x} \in R^{601}$. The cosine similarity term provides an explicit interaction signal capturing the degree of semantic alignment between query and passage, complementing the individual embedding components which carry no cross-text information on their own.

A logistic regression classifier was implemented from scratch using mini-batch SGD (batch size 512) with *weighted* binary cross-entropy loss. To address the severe class imbalance ($\approx 0.1\%$ positive rate, with $\approx 4{,}590$ relevant passages from $4{,}364{,}339$ total training rows), a positive-class weight of $w^+ = 10$ is applied to the BCE gradient, amplifying the contribution of relevant passages relative to non-relevant ones by a factor of ten. The model learns weights $\mathbf{w} \in R^{601}$ and bias b via the update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$, where $\mathcal{L}$ is the mean weighted BCE over a batch. To avoid materialising the large feature matrix in RAM, passage embeddings are computed once per unique passage ID and the full feature matrix is written to a temporary memory-mapped file to allow training on the full dataset. SGD iterates over this file sequentially without shuffling between epochs (avoiding slow random disk access); with 4.3M samples the sequential batches are sufficiently diverse. In the ranking stage, passages are ordered by their predicted positive-class probability $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ rather than by a hard threshold decision, so the model functions as a relevance ranker rather than a binary classifier at test time. Table 3 reports the learning rate analysis (2 epochs each).

η	Train BCE	Val MAP	Val NDCG@10
10^{-4}	0.3092	0.0133	0.0125
10^{-3}	0.0776	0.0134	0.0126
10^{-2}	0.0614	0.0133	0.0127
10^{-1}	0.0611	0.0137	0.0133
3×10^{-1}	0.0607	0.0268	0.0252

Table 3: Learning rate analysis (2 epochs each, batch size 512, $w^+ = 10$).

At small learning rates ($\eta \leq 10^{-2}$), the model converges to low BCE but achieves near-identical MAP ($\approx$0.0133–0.0134), indicating that gradient steps are too small to meaningfully reweight the classifier away from the majority-class prior within 2 epochs. At $\eta = 10^{-1}$ MAP improves modestly to 0.0137, and at $\eta = 3 \times 10^{-1}$ a substantial jump to 0.0268 is observed, demonstrating that a sufficiently large learning rate allows the weighted gradient signal from the rare positive examples to propagate meaningfully through the parameters before convergence stalls. BCE plateaus at 0.0607 across the two largest learning rates, confirming that the MAP improvement at $\eta = 0.3$ is not an artefact of further loss reduction but rather a genuine improvement in ranking quality driven by the positive-class weighting. The best learning rate $\eta = 0.3$ is selected; the final model trained for 3 epochs achieves MAP$= 0.0302$, NDCG@10$= 0.0287$.

The logistic regression outperforms both BM25 variants (MAP 0.0302 vs 0.0193). This improvement is attributable to two complementary mechanisms. First, the weighted BCE loss ($w^+ = 10$) ensures that the rare relevant passages contribute meaningfully to the gradient, preventing the model from collapsing to a trivial all-irrelevant solution. Second, the cosine similarity interaction feature provides a direct measure of semantic alignment between query and passage embeddings, giving the classifier a compact and informative relevance signal that the concatenated embeddings alone do not expose. For paraphrased queries (e.g. *"what slows blood flow"* matched against a passage using *"reduces circulation"*), this semantic signal allows the model to retrieve topically relevant passages that share no exact terms with the query, which BM25 would fail to reward. The residual gap between

the embedding model and a hypothetical upper bound reflects the information loss from averaging word vectors, which discards positional and term-specificity information that BM25 captures via IDF weighting. For highly specific queries such as *"legionella pneumophila cause pneumonia"*, BM25 still rewards exact term matches with high IDF (e.g. $\mathrm{df} \approx 14$ for *legionella*), whereas averaged FastText embeddings may conflate semantically proximate but non-matching terms, losing discriminative precision.